

Computer Science

04. CPU Scheduling

4주차: CPU 스케줄링

오늘의 핵심 질문



동시 실행의 비밀

"게임을 하면서 유튜브 음악을 틀고 웹서핑을 동시에 할 수 있는 이유는?"

실제 CPU 코어 개수는 한정되어 있지만, 사용자는 수십 개의 앱이 순간적으로 동시에 실행되는 것처럼 느낌



자원 배분의 우선순위

"대기열에 100개 프로세스가 있을 때, CPU는 누구를 실행할까?"

빠른 응답이 필요한 작업과 장시간 계산이 필요한 작업 사이에서 운영체제가 중재하는 규칙은?

목차 INDEX

01 CPU Scheduling

- CPU Scheduler
- Ready Queue
- CPU Burst
- Preemptive

02 스케줄링 평가기준

- Utilization
- Throughput
- 반응 시간

03 스케줄링 알고리즘

- FCFS
- SJF & SRTF
- Round Robin

01

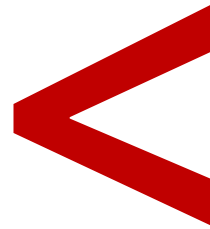
CPU Scheduling

CPU 스케줄링

01 CPU Scheduling

핵심 문제

기본 속도:	2.50GHz
소켓:	1
코어:	24
논리 프로세서:	24
가상화:	사용
L1 캐시:	2.4MB
L2 캐시:	40.0MB
L3 캐시:	36.0MB



이름	PID	스레드
explorer.exe	29528	314
firefox.exe	4788	29
firefox.exe	27296	34
firefox.exe	10708	30
firefox.exe	24320	92
firefox.exe	21756	215
firefox.exe	6488	5
firefox.exe	17700	28
firefox.exe	21440	33
firefox.exe	32868	28
firefox.exe	16264	4
firefox.exe	35360	32
firefox.exe	28764	30
firefox.exe	6984	30
firefox.exe	17728	4
firefox.exe	25392	28
firefox.exe	5024	26
firefox.exe	13296	26
firefox.exe	19012	5

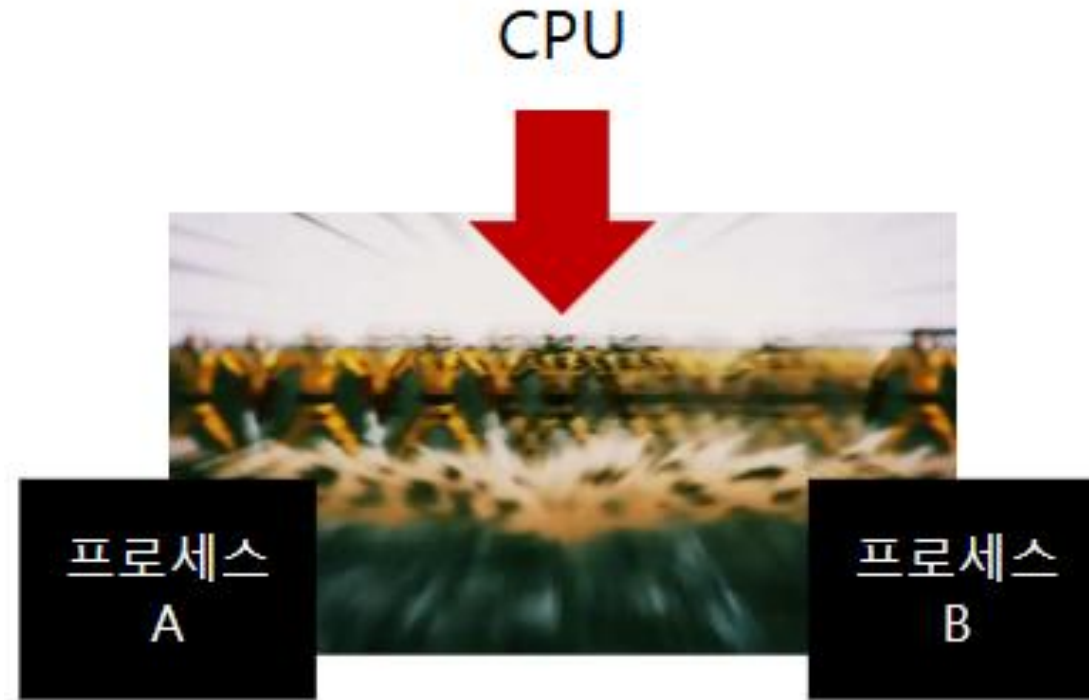
코어 수: 24

Firefox + 탐색기의 총 스레드: 993개

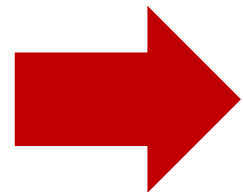
코어 하나는 한번에 스레드 1개 실행 가능

01 CPU Scheduling

해결 아이디어



이렇게 프로세스를 순식간에 바꿔주면 되지 않을까?
근데 어느 프로세스에 어느 코어 할당할지 누가 정해줌?



CPU 스케줄러

01 CPU Scheduling

프로세스의 3대 상태

▶ Running & Ready 상태

Running (실행):

프로세스가 CPU를 할당받아 실제로 명령어를 수행 중인 상태

Ready (준비):

CPU만 할당받으면 즉시 실행할 수 있게 준비가 다 끝난 상태로

Ready Queue에서 대기

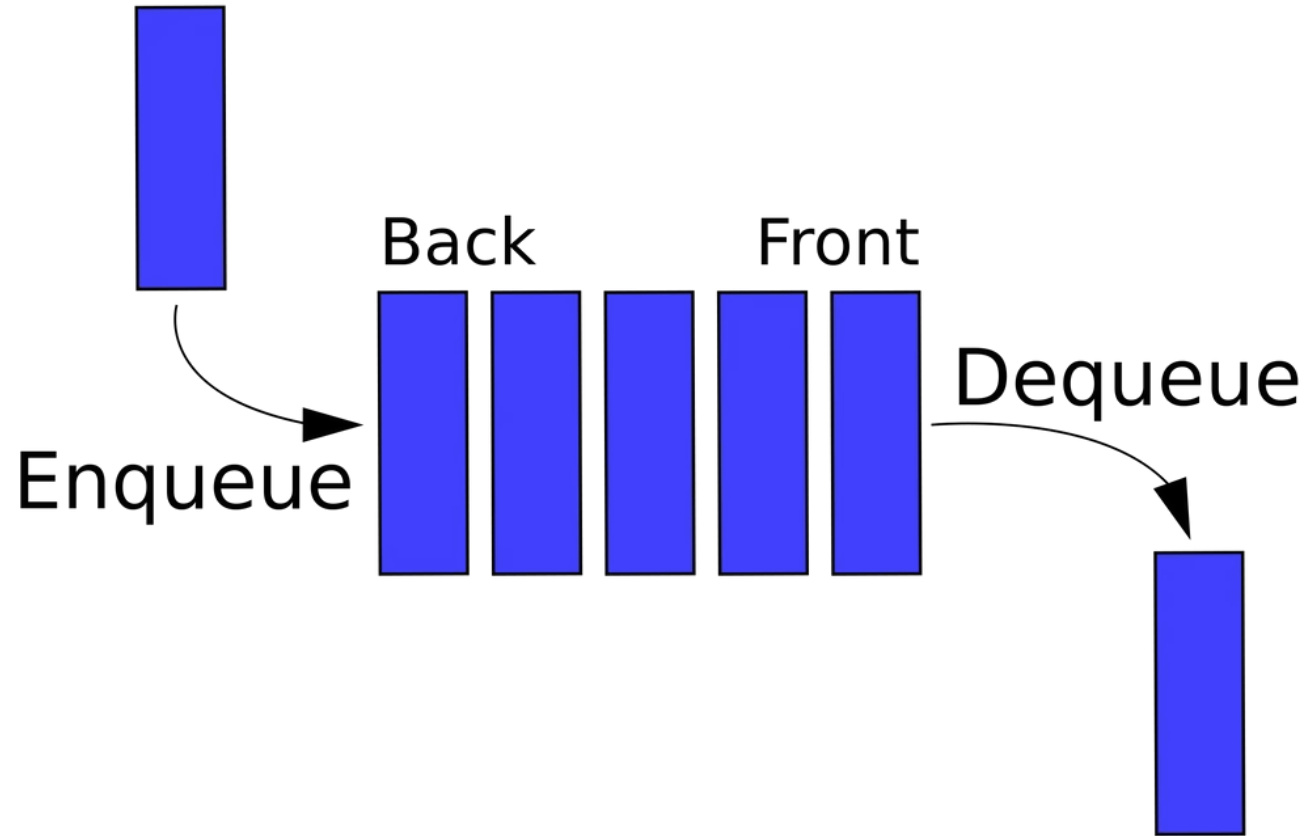
|| Waiting (Blocked) 상태

Waiting (대기):

입출력(I/O) 작업 완료나 키보드 입력, 파일 읽기 등의 이벤트 발생을 기다리는 일시 중단 상태

01 CPU Scheduling

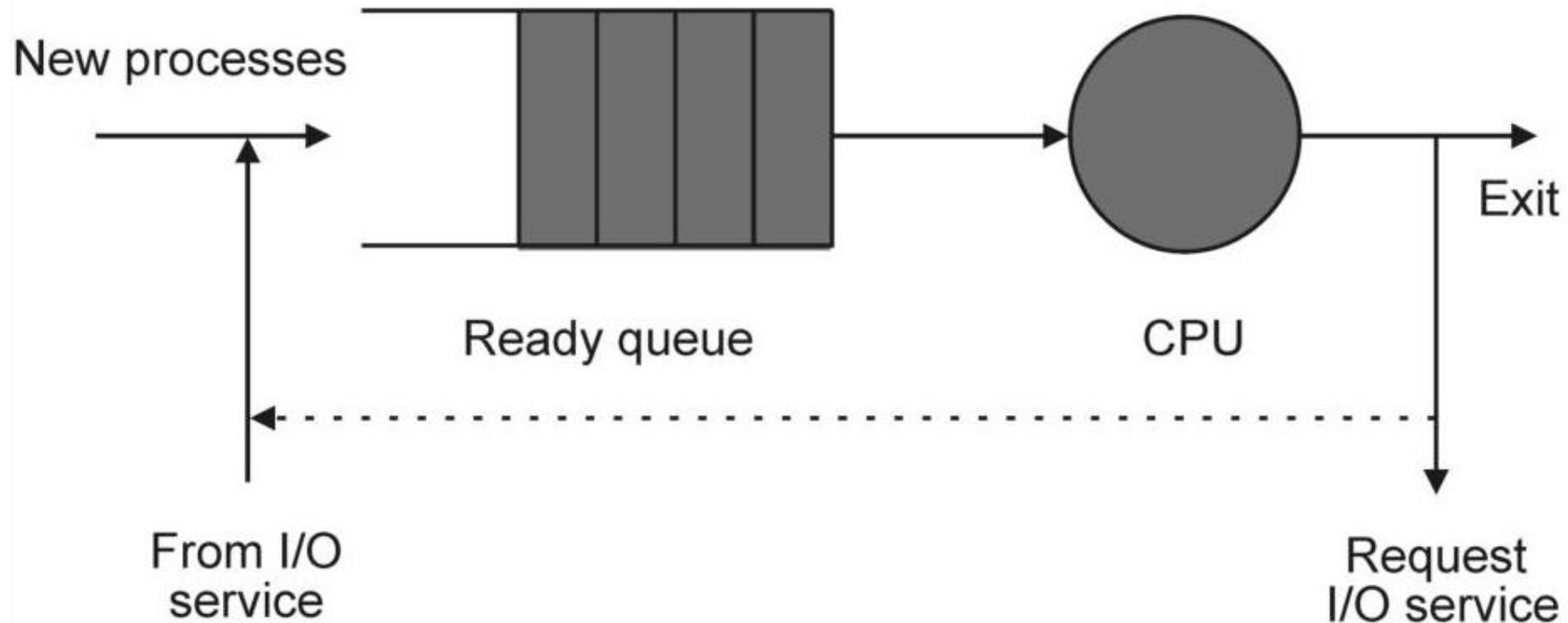
Ready Queue



큐(Queue)는 이렇게 생김
먼저 들어온 게 먼저 나가는 구조임 (FIFO)

01 CPU Scheduling

Ready Queue



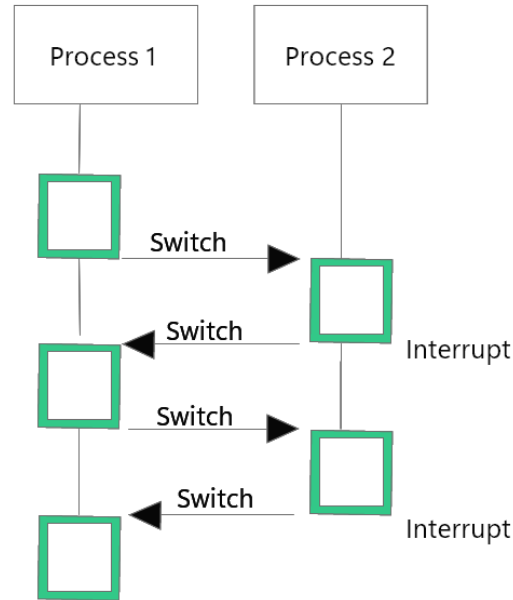
실행될 프로세스를 큐에 모아놓음
그런 다음 선착순으로 실행
그 큐를 Ready Queue라고 함

01 CPU Scheduling

Preemptive & Non-Preemptive

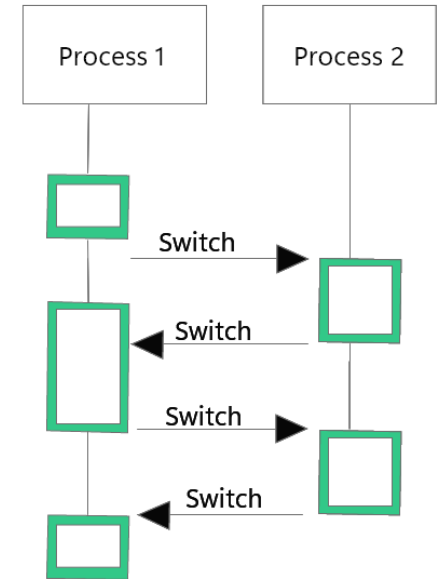
- CPU 자원을 받은 프로세스는 종료되거나 대기 상태에 들어가면 할당받은 CPU를 놓아줌
- 이렇게 프로세스가 CPU를 놓아줬을때만 다른 프로세스로 할당시키는걸 비선점, Non-Preemptive라고 함
- 반면 OS가 실행 중인 프로세스의 CPU를 강제로 뺏아서 다른 프로세스에 할당하는 경우도 있음
- 이를 선점, Preemptive라고 함

Preemptive Scheduling



vs

Non-Preemptive Scheduling



01 CPU Scheduling

CPU Burst & I/O Burst

CPU Burst (연산 구간)

프로세스가 실제로 코어를 할당받아서 실행되고 있는 시간

CPU-bound Process: 행렬 연산, 3D 렌더링, 영상 인코딩 등 CPU Burst가 매우 긴 프로그램

I/O Burst (입출력 대기)

CPU가 할 일이 없어 입력·출력을 기다리는 시간
(파일 읽기, 키보드 입력 대기 등)

I/O-bound Process: 문서 편집기, 웹 브라우저, 채팅 앱 등 I/O Burst가 자주 발생하는 프로그램

01 CPU Scheduling

CPU Burst & I/O Burst

CPU Burst (연산 구간)

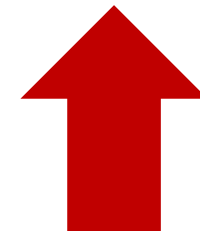
프로세스가 실제로 코어를 할당받아서 실행되고 있는 시간

CPU-bound Process: 행렬 연산, 3D 렌더링, 영상 인코딩 등 CPU Burst가 매우 긴 프로그램

I/O Burst (입출력 대기)

CPU가 할 일이 없어 입력·출력을 기다리는 시간
(파일 읽기, 키보드 입력 대기 등)

I/O-bound Process: 문서 편집기, 웹 브라우저, 채팅 앱 등 I/O Burst가 자주 발생하는 프로그램



이럴 때 CPU에게 다른 작업 시키면 되지 않을까?

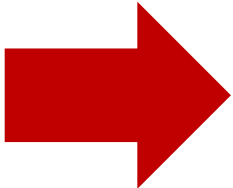
02

스케줄링 평가기준

Scheduling Criteria

스케줄링의 5대 평가 지표

지표명 (Metric)	영문 명칭	의미 및 정의	최적화 목표
CPU 이용률	CPU Utilization	전체 시간 중 CPU가 실제로 가동된 시간의 비율 (%)	최대화 (Maximize)
처리량	Throughput	단위 시간당 완료된 프로세스의 개수	최대화 (Maximize)
반환 시간	Turnaround Time	프로세스가 Ready Queue에 들어와서 완전히 끝날 때까지의 총 시간	최소화 (Minimize)
대기 시간	Waiting Time	프로세스가 Ready Queue에서 CPU 할당을 기다린 총 시간	최소화 (Minimize)
응답 시간	Response Time	대기열 진입 후 CPU를 처음으로 할당받기까지 걸린 시간	최소화 (Minimize)



이걸 최적화하는 알고리즘이 스케줄링 알고리즘

03

스케줄링 알고리즘

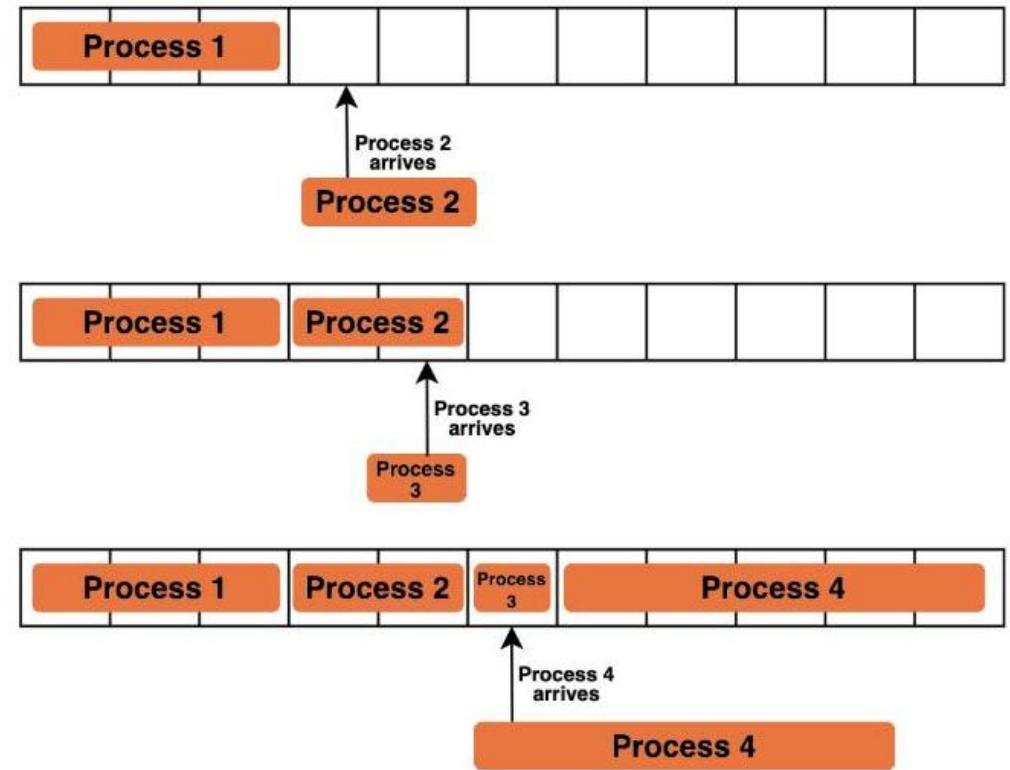
Scheduling Algorithm

03 스케줄링 알고리즘

FCFS: First Come, First Served

- 선착순 방식
- Non-Preemptive 방식
- 구현이 매우 단순함 (단순 Queue)
- 한번 CPU를 잡으면 프로세스가 놓기 전까지 뺏기지 않음
- 모든 프로세스가 언젠가는 실행됨
- 긴 실행 시간의 프로세스 뒤에 있는 프로세스는 한참 기다려야 하는 단점이 있음

First Come First Serve



03 스케줄링 알고리즘

FCFS: First Come, First Served

[예제 조건] 프로세스 도착 순서: P1 (24ms) → P2 (3ms) → P3 (3ms) (모두 0ms 도착)



대기 시간 계산: P1 = 0ms, P2 = 24ms, P3 = 27ms

평균 대기 시간 (Average Waiting Time): $(0 + 24 + 27) / 3 = 17.0 \text{ ms}$

03 스케줄링 알고리즘

FCFS: First Come, First Served

Case A: P1 → P2 → P3

긴 프로세스가 먼저 온 경우:

- P1 대기시간: 0ms
- P2 대기시간: 24ms
- P3 대기시간: 27ms

평균 대기시간 = 17.0 ms

Case B: P2 → P3 → P1

짧은 프로세스가 먼저 온 경우:

- P2 대기시간: 0ms
- P3 대기시간: 3ms
- P1 대기시간: 6ms

평균 대기시간 = 3.0 ms (82% 감소!)

선착순으로 하면 성능이 너무 운빨일듯

03 스케줄링 알고리즘

SJF: Shortest Job First

[예제] P1(6ms), P2(8ms), P3(7ms), P4(3ms) 모두 0ms 도착

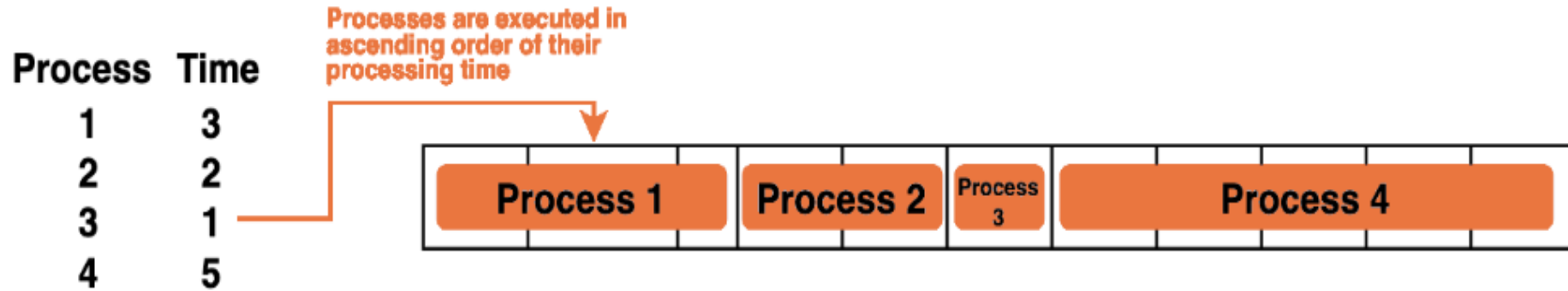


SJF 대기 시간: P4=0ms, P1=3ms, P3=9ms, P2=16ms → 평균 대기 시간: 7.0 ms
(참고: FCFS 적용 시 P1→P2→P3→P4 순서로 평균 대기 시간 10.25 ms 발생)

실행 시간 짧은 놈부터 실행
(실행시간 똑같으면 먼저 온 놈부터)
아도 Non-Preemptive 방식임

SJF 의 문제

Shortest Job First

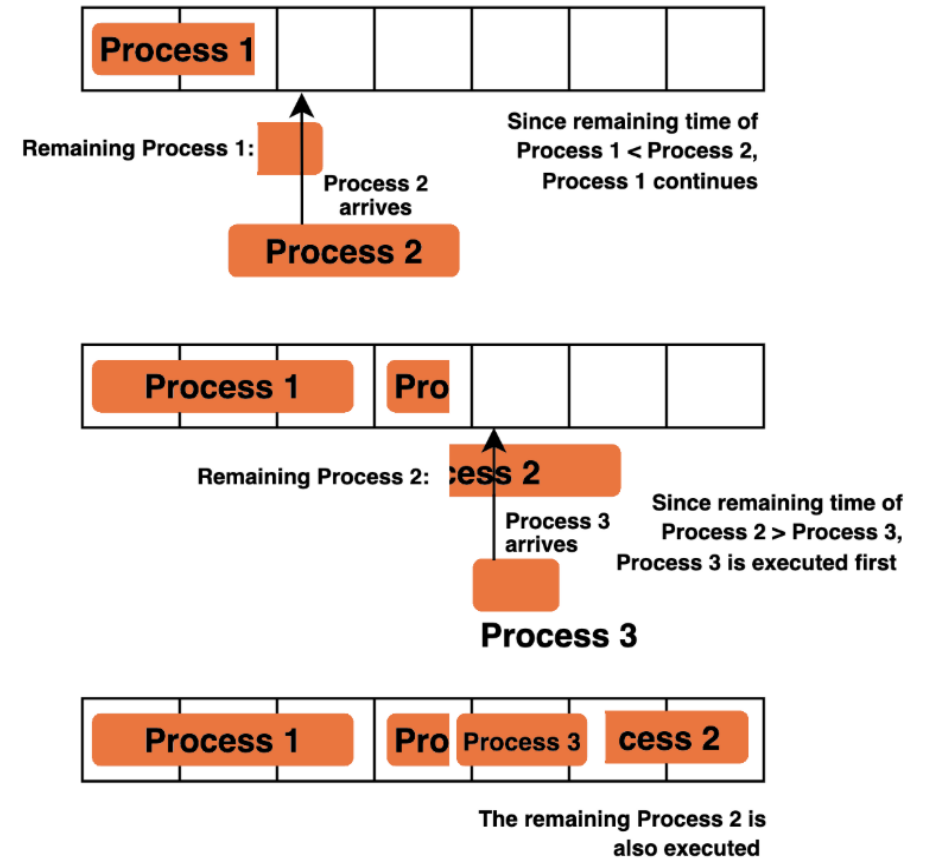


1. 실행 시간 정확히 얼마나 걸릴지 어떻게 미리 알지?
2. 짧은 프로세스만 계속 들어오면 긴 프로세스는 CPU 못잡는거 아닌가?

SRTF: Shortest Remaining Time First

- 아까 배운 SJF의 Preemptive 버전임
- 정확한 실행 과정을 알아봅시다
 1. 새로운 프로세스가 도착함
 2. 그 프로세스의 실행 시간이 현재 실행 중인 프로세스의 남은 시간보다 짧으면 CPU를 빼앗아 이거 먼저 함
 3. 만약 더 길다면 그냥 기다린 후 실행
- 다만 프로세스가 들어올때마다 남은 시간을 계산해야 하고, Context Switching이 많이 일어나서 시간이 좀 더 걸릴 수도 있음
- 그리고 프로세스 실행 시간 어떻게 정확히 계산함? (SJF와 동일한 문제를 공유)

Shorted Remaining Time First

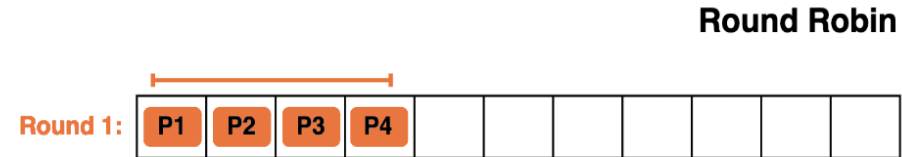


03 스케줄링 알고리즘

Round Robin

- 이번엔 CPU를 쓸 수 있는 시간을 제한해봄
- 그걸 Time Quantum이라 함
- 프로세스가 CPU를 사용하며 실행되다가 Time Quantum이 지나면 CPU 할당을 빼앗기고 Ready Queue 맨 뒤로 강제 이동됨
- Time Quantum은 보통 수 ms ~ 수십 ms 정도 됨
- Time Quantum이 너무 짧으면 Context Switching 비용이 폭발하고, 너무 길면 FCFS랑 비슷해져서 성능이 좀 떨어짐
(트레이드오프를 고려해서 설계해야됨)
- 현대에는 대부분 이 방식 사용함

Process	Time
1	3
2	2
3	1
4	5



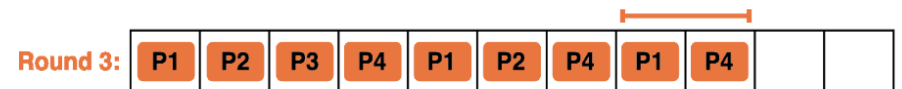
Remaining Status of each Process after Round 1:

Process	Time
1	$3 - 1 = 2$
2	$2 - 1 = 1$
3	$1 - 1 = 0$
4	$5 - 1 = 4$



Remaining Status of each Process after Round 2:

Process	Time
1	$2 - 1 = 1$
2	$1 - 1 = 0$
3	0
4	$4 - 1 = 3$



Remaining Status of each Process after Round 3:

Process	Time
1	$1 - 0 = 0$
2	$1 - 1 = 0$
3	0
4	$3 - 1 = 2$

